



30-Day Android Challenge

Mastering Android Programming Interviews Globally
(Junior-Mid-Senior-Architect-Level Deconstruction 📱)

Day 26: Advanced Kotlin Features



Esmaeeil Moradi
@esmaeeil-moradi700

Day 26



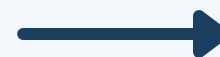
**of #30DayAndroidChallenge:
Comprehensive Answers for "Day 26: Advanced Kotlin
Features" - #30DayAndroidChallenge**

Hello to all my dear friends and companions! 🙌

Welcome to Day 26 of the #30DayAndroidChallenge! Today, we're taking a deep dive into the advanced features of Kotlin that go beyond the basics. Understanding these concepts is what elevates a developer from a good one to a great one, allowing you to write more expressive, concise, and powerful code.



Esmaeeil Moradi
@esmaeeil-moradi700



Junior Level Questions



Esmaeeil Moradi
@esmaeeil-moradi700



Explain Kotlin coroutines context and dispatchers.

Coroutine context is a set of data that defines the behavior of a coroutine. Think of it as a collection of elements, each providing a specific characteristic to the coroutine. The most important element in the context is the Job, which controls the coroutine's lifecycle, and the Dispatcher, which determines the thread(s) the coroutine will run on.

Dispatchers are a key part of the context. They are responsible for dispatching the coroutine's execution to a specific thread pool.

Kotlin provides three main dispatchers for common use cases:



Esmaeel Moradi

@esmaeeil-moradi700



- **Dispatchers.Main:** This dispatcher is tied to the main thread of the Android UI. It should be used for any UI-related work, such as updating views or handling user interactions. Since it's a single thread, any heavy work on this dispatcher will freeze the UI.
- **Dispatchers.IO:** This dispatcher is optimized for I/O-bound tasks, which are operations that involve a lot of waiting (e.g., waiting for a network response, reading from a file, or database access). It uses a shared pool of threads, which is more efficient for blocking operations than Dispatchers.Default.
- **Dispatchers.Default:** This dispatcher is designed for CPU-intensive tasks, such as sorting large lists, complex calculations, or image processing. It uses a shared pool of threads, and the number of threads is limited by the number of CPU cores.



Esmaeel Moradi

@esmaeel-moradi700

Code Example:



```
import kotlinx.coroutines.*

// This is a suspend function that can be paused and resumed.
suspend fun fetchData(): User {
    // This coroutine runs on the I/O thread pool.
    return withContext(Dispatchers.IO) {
        // Simulating a network call
        delay(1000)
        User("Esmaeel", 35)
    }
}

fun updateUI(user: User) {
    // This coroutine runs on the Main thread.
    // It's safe to update the UI from here.
    println("Updating UI with user: ${user.name}")
}

fun main() {
    // Start a new coroutine on the main thread
    GlobalScope.launch(Dispatchers.Main) {
        // Call a suspend function
        val user = fetchData()
        // Switch back to the main thread to update the UI
        updateUI(user)
    }
}
```



Esmaeel Moradi
@esmaeel-moradi700



The withContext function is a powerful tool that allows you to switch the coroutine's context to a different dispatcher and then automatically switch back when the block is finished.



Esmaeel Moradi
@esmaeel-moradi700



What are inline functions and when to use them?

An **inline function** is a function that the compiler copies and pastes directly into the calling site instead of making a function call. This eliminates the overhead of function calls, which includes creating a new stack frame and passing arguments.

How it works: The Kotlin compiler replaces the function call with the actual body of the function during compilation.

When to use them: The primary use case for inline is with **higher-order functions**—functions that take other functions (lambdas) as arguments. Lambdas in Kotlin are objects, and creating these objects for every function call can incur a performance penalty. By inlining the function, the lambda is also inlined, and no object is created, leading to a significant performance improvement.



Esmaeel Moradi

@esmaeel-moradi700

Code Example:



```
// A regular higher-order function
fun measureTime(block: () -> Unit) {
    val start = System.nanoTime()
    block()
    val end = System.nanoTime()
    println("Time taken: ${end - start} ns")
}

// An inline higher-order function
inline fun measureTimeInline(block: () -> Unit) {
    val start = System.nanoTime()
    block()
    val end = System.nanoTime()
    println("Time taken: ${end - start} ns")
}

fun main() {
    measureTime {
        // code to measure
    }

    measureTimeInline {
        // code to measure
    }
}
```



Esmaeel Moradi
@esmaeeil-moradi700



Trade-offs:

While inline can improve performance, it also increases the size of the compiled bytecode because the function's code is duplicated wherever it's called. Therefore, you should use inline judiciously, typically only for small, performance-critical higher-order functions.



Esmaeel Moradi

@esmaeeil-moradi700



How do you use Kotlin delegation patterns?

Kotlin delegation is a language feature that allows an object to delegate part of its functionality to another object. This pattern helps to avoid boilerplate code and promotes code reuse. Kotlin supports two main types of delegation.



Esmaeel Moradi
@esmaeeil-moradi700

1. Property Delegation (by):

This is a powerful pattern that allows you to delegate the getter and setter logic of a property to a separate class.

Kotlin provides several built-in property delegates.

- **lazy**: The value of the property is computed only on its first access.

```
val myVariable: String by lazy {
    println("Computing value...")
    "This is a lazy value"
}

fun main() {
    println(myVariable) // "Computing value..." is printed only here.
    println(myVariable) // The value is already computed.
}
```

- **Delegates.observable**: Notifies you every time the property's value changes.

```
import kotlin.properties.Delegates

var name: String by Delegates.observable("Initial name") {
    property, oldValue, newValue ->
    println("Name changed from $oldValue to $newValue")
}

fun main() {
    name = "Esmaeeil" // "Name changed from Initial name to Esmaeeil" is printed.
}
```

You can also create custom property delegates to implement your own logic.



Esmaeeil Moradi
@esmaeeil-moradi700



2. Class Delegation (by):

This pattern allows a class to implement an interface by delegating the implementation of its methods to another object. This is an elegant alternative to manually writing wrapper classes.



Esmaeiel Moradi
@esmaeiel-moradi700

Code Example:



```
interface Repository {
    fun saveData(data: String)
    fun fetchData(): String
}

class SQLiteRepository : Repository {
    override fun saveData(data: String) {
        println("Saving data to SQLite: $data")
    }

    override fun fetchData(): String {
        return "Data from SQLite"
    }
}

// The cache repository delegates all calls to the SQL repo
// except for the ones it wants to handle itself.
class CachingRepository(private val sqlRepo: SQLiteRepository) : Repository by sqlRepo {
    private val cache = mutableMapOf<String, String>()

    override fun fetchData(): String {
        return cache.getOrPut("key") {
            println("Fetching data from SQL and caching it...")
            sqlRepo.fetchData()
        }
    }
}

fun main() {
    val cachingRepo = CachingRepository(SQLiteRepository())
    cachingRepo.saveData("Hello") // Delegated to SQLiteRepository
    println(cachingRepo.fetchData()) // Fetch from SQL and cache
    println(cachingRepo.fetchData()) // Fetch from cache
}
```



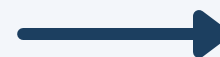
Esmaeel Moradi
@esmaeeil-moradi700



The CachingRepository class uses by sqlRepo to delegate the saveData method directly to SqlLiteRepository, while it overrides the fetchData method to add its own caching logic. This pattern greatly reduces boilerplate code.



Esmaeel Moradi
@esmaeeil-moradi700



Mid-Level Questions



Esmaeel Moradi
@esmaeeil-moradi700

Implement custom DSLs using Kotlin language features.

A Domain-Specific Language (DSL) is a programming language tailored to a specific application domain. In Kotlin, you can create powerful, readable, and type-safe DSLs that look and feel like natural language.

Key Kotlin features for building a DSL:

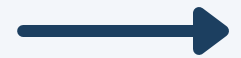
1. **Lambda with Receiver:** A lambda that can access the members of an object without needing to use this or the object's name. This is the foundation of Kotlin DSLs.
2. **Infix Functions:** Functions that can be called without a dot and parentheses. They make the code more readable (e.g., a to b).
3. **Extension Functions:** Allow you to add new functions to existing classes without inheriting from them.



Esmaeel Moradi

@esmaeel-moradi700

Practical Code Example: A simple HTML-like DSL



```
// 1. A builder class with a receiver
class HtmlBuilder {
    private val children = mutableListOf<String>()

    // 2. An extension function on the builder
    fun p(text: String) {
        children.add("<p>$text</p>")
    }

    // 3. A function that takes a lambda with a receiver
    fun body(block: BodyBuilder.() -> Unit) {
        val bodyBuilder = BodyBuilder()
        bodyBuilder.block()
        children.add("<body>\n${bodyBuilder.build()}\n</body>")
    }

    fun build() = "<html>\n${children.joinToString("\n")}\n</html>"
}

class BodyBuilder {
    private val children = mutableListOf<String>()
    fun build() = children.joinToString("\n")
    fun h1(text: String) = children.add("<h1>$text</h1>")
    fun p(text: String) = children.add("<p>$text</p>")
}

// A top-level function to start the DSL
fun html(block: HtmlBuilder.() -> Unit): String {
    val builder = HtmlBuilder()
    builder.block()
    return builder.build()
}

fun main() {
    val htmlContent = html {
        body {
            h1("Welcome to my website!")
            p("This is a paragraph.")
        }
    }
    println(htmlContent)
}
```



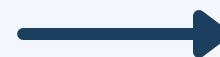
Esmaeel Moradi
@esmaeel-moradi700



In this example, the html and body functions take a lambda with a receiver. This allows you to write code that looks like a nested HTML structure, making it highly readable and intuitive. This concept is widely used in frameworks like Gradle's Groovy DSL and Jetpack Compose.



Esmaeeil Moradi
@esmaeeil-moradi700



How do you use Kotlin reflection effectively?

Reflection is the ability of a program to examine, introspect, and modify its own structure and behavior at runtime. It allows you to inspect classes, properties, and functions and even call them dynamically.

When to use it:

- **Serialization/Deserialization:** Libraries like Moshi or `kotlinx.serialization` use reflection (or a compiler plugin) to automatically map JSON data to Kotlin classes without needing to write boilerplate code.
- **Annotation Processing:** Frameworks like Dagger or Spring use reflection to find and process annotations on classes to generate code or configure components.
- **Library Development:** To create flexible APIs that can work with any class, reflection is often necessary.



Esmaeel Moradi

@esmaeel-moradi700

Performance Implications:

Reflection is an expensive operation because it's not a direct function call but involves searching for the class and its members at runtime. It's significantly slower than a regular function call. Therefore, it should be used judiciously and avoided in performance-critical code paths, such as the `onDraw` method of a view.

Safe, Simple Use-Case Example: A common, safe use case is to inspect a data class and print all its properties.

```
data class User(val name: String, val age: Int)

fun printProperties(obj: Any) {
    // Get the KClass from the object instance
    val kClass = obj::class

    println("Properties of class ${kClass.simpleName}:")

    // Iterate over the properties of the class
    kClass.members.forEach { member ->
        // Check if the member is a property and not a function
        if (member is kotlin.reflect.KProperty1<*, *>) {
            // Get the value of the property for the given object
            val value = member.getter.call(obj)
            println("${member.name}: $value")
        }
    }
}

fun main() {
    val user = User("Esmaeel", 35)
    printProperties(user)
}
```



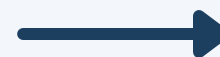
Esmaeel Moradi
@esmaeeil-moradi700



This example shows how to use reflection to introspect an object and print its properties without knowing the class structure at compile time.



Esmaeel Moradi
@esmaeeil-moradi700



Design type-safe builders and configuration DSLs.

Type-safe builders are a more advanced form of DSLs that use the Kotlin compiler's type system to prevent incorrect usage at compile time. They ensure that the resulting structure is valid without needing to check for runtime errors.

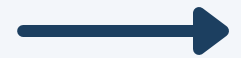
Core concepts:

- **Lambda with Receiver:** The same as in basic DSLs.
- **@DslMarker Annotation:** This annotation is crucial. It prevents unintended scope pollution. When applied to the builder's marker class, it restricts a lambda from accessing members of outer builders. This ensures that you can't, for example, add a button inside a row in a different scope.



Esmaeel Moradi

@esmaeel-moradi700



Concrete Example: A UI builder (like a simplified Jetpack Compose-like DSL)

```
// Marker annotation to create a type-safe DSL
@DslMarker
annotation class UiDsl

@UiDsl
class ColumnBuilder {
    private val children = mutableListOf<String>()

    fun text(value: String) {
        children.add("Text(\\"$value\\")")
    }

    fun build(): String = "Column(\n${children.joinToString("\n")}\n)"
}

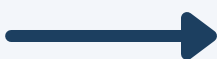
@UiDsl
class RowBuilder {
    private val children = mutableListOf<String>()

    fun text(value: String) {
        children.add("Text(\\"$value\\")")
    }

    fun build(): String = "Row(\n${children.joinToString("\n")}\n)"
}
```



Esmaeel Moradi
@esmaeeil-moradi700



```
// Top-level entry points for the DSL
fun Column(block: ColumnBuilder() -> Unit): String {
    val builder = ColumnBuilder()
    builder.block()
    return builder.build()
}

fun Row(block: RowBuilder() -> Unit): String {
    val builder = RowBuilder()
    builder.block()
    return builder.build()
}

fun main() {
    val uiCode = Column {
        Row {
            text("First item")
            text("Second item")
        }
        text("A third item")
        // If you remove @DslMarker, you could call
        // Column { ... } here, which is semantically incorrect.
        // The annotation prevents this.
    }

    println(uiCode)
}
```



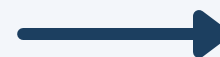
Esmaeel Moradi
@esmaeeil-moradi700



By adding the `@DslMarker` annotation, the compiler prevents you from accidentally calling `Column` inside a `Row`'s lambda, enforcing a clear and correct structure. This is the core principle behind the safety and expressiveness of frameworks like Jetpack Compose.



Esmaeel Moradi
@esmaeeil-moradi700



Senior Level Questions



Esmaeeil Moradi
@esmaeeil-moradi700



Create advanced Kotlin abstractions for domain-specific problems.

Senior developers use advanced Kotlin features to model complex domain problems in a clear, concise, and type-safe manner. This makes the code less error-prone and easier to understand.

Key features for advanced abstractions:

1. **Sealed Classes:** Used to represent a restricted class hierarchy. All subclasses are known at compile-time, which allows for exhaustive when expressions, eliminating the need for an else branch. This is perfect for modeling a state machine.
2. **Inline Classes:** Used to create a type-safe wrapper around a primitive type without incurring runtime overhead. They are useful for creating "strong types" for values like UserId or EmailAddress.
3. **Extension Functions:** Allow you to add new behavior to existing classes, making APIs more fluent and readable.



Esmaeel Moradi

@esmaeel-moradi700

Complex Example: A state machine for a network request

```
// 1. Sealed class to model the states of a network request
sealed class ApiState {
    object Idle : ApiState()
    object Loading : ApiState()
    data class Success(val data: String) : ApiState()
    data class Error(val message: String) : ApiState()
}

// 2. Inline class for a type-safe user ID
@JvmInline
value class UserId(val value: String)

// 3. Extension function to add behavior to the sealed class
fun ApiState.isFinished(): Boolean {
    return this is ApiState.Success || this is ApiState.Error
}

fun processApiState(state: ApiState) {
    when (state) {
        ApiState.Idle -> println("Request is idle.")
        ApiState.Loading -> println("Request is loading...")
        is ApiState.Success -> println("Request successful with data: ${state.data}")
        is ApiState.Error -> println("Request failed with message: ${state.message}")
    }
}

fun main() {
    val successState = ApiState.Success("User data fetched successfully")
    processApiState(successState)

    // Using the extension function
    println("Is request finished? ${successState.isFinished()}")

    // Using the inline class
    val userId = UserId("user_12345")
    println("User ID: ${userId.value}")
}
```



Esmaeel Moradi
@esmaeeil-moradi700



This example shows how a combination of these features can be used to create an expressive and robust system. The when expression on a sealed class is exhaustive, meaning the compiler will warn you if you forget a state. The inline class prevents a simple string from being passed where a UserId is expected, preventing a whole class of bugs.



Esmaeel Moradi
@esmaeeil-moradi700

Implement performance-critical code using advanced Kotlin features.

Writing performance-critical code in Kotlin involves leveraging features that reduce object allocation, memory usage, and CPU cycles.

1. Inline Classes:

- **Concept:** As discussed, inline classes wrap a primitive type but are compiled to the primitive type itself.
- **Performance Benefit:** They are excellent for creating type-safe wrappers around primitive types (Int, String, etc.) without the performance penalty of creating a new object. This reduces memory allocation and garbage collection pressure.
- **Example:** Using an inline class for an Amount instead of a regular data class with a Double.



Esmaeel Moradi

@esmaeel-moradi700



2. Operator Overloading:

- **Concept:** Allows you to redefine the behavior of operators (+, -, *, etc.) for your custom classes.
- **Performance Benefit:** When used with inline classes, this can lead to highly performant, readable code. For example, you can define addition for two Amount inline classes, and the compiler will generate primitive-level arithmetic, which is very fast.
- **Pitfalls:** Operator overloading can be confusing if not used intuitively. Avoid using it for operations that don't make logical sense.

3. inline and noinline for Lambdas:

- **Concept:** You can use noinline on a lambda parameter of an inline function to prevent that specific lambda from being inlined.
- **Performance Benefit:** This is useful when you need to pass a lambda from an inline function to another function that is not inlined. It prevents the creation of a lambda object unnecessarily.



Esmaeel Moradi

@esmaeel-moradi700



Example: A performant Amount class

```
@JvmInline
value class Amount(val value: Double) {
    operator fun plus(other: Amount): Amount {
        return Amount(this.value + other.value)
    }
}

fun main() {
    val amount1 = Amount(10.5)
    val amount2 = Amount(20.0)

    val total = amount1 + amount2 // Using the overloaded '+' operator
    println("Total amount: ${total.value}")

    // The compiler will optimize this to primitive addition, avoiding object allocation.
}
```



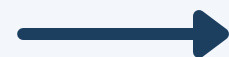
Esmaeel Moradi
@esmaeel-moradi700



By carefully using these features, a senior developer can write code that is both highly readable and performs as well as, or even better than, its Java counterpart.



Esmaeel Moradi
@esmaeeil-moradi700



How would you design Kotlin-based frameworks and libraries?

Designing a framework or a reusable library requires careful architectural planning to ensure it is robust, extensible, and easy to use.

- **1.API Design (Fluent and Idiomatic Kotlin):**
 - **Prioritize Immutability:** Use `val` and data class for data holders. This makes the API predictable and thread-safe.
 - **Use Extension Functions:** Add functionality to existing classes to create a more fluent API. This is especially useful for creating DSLs.
 - **Favor Coroutines:** For asynchronous APIs, design them to be coroutine-friendly using suspend functions and Flow.



Esmaeel Moradi
@esmaeel-moradi700

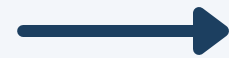


- **2.Extensibility Points:**
 - **Interfaces and Abstract Classes:** Define clear interfaces that users can implement to extend your library's functionality.
 - **Sealed Classes:** Use sealed classes to define a closed set of extension points.
 - **Type-Safe Builders:** Design APIs using type-safe builders for a clean and intuitive configuration.
- **3.Encapsulation and Hiding Implementation Details:**
 - **internal keyword:** Use the internal visibility modifier for classes, functions, and properties that should only be accessible within your library module. This prevents users from depending on internal implementation details, which gives you the freedom to refactor the internals without breaking the public API.
 - **Private APIs:** Use private and protected to hide internal logic.



Esmaeel Moradi

@esmaeel-moradi700



- **4.Documentation and Examples:**

- A well-designed library is useless without good documentation. Provide clear, concise documentation for every public API.
- Include practical, runnable examples to demonstrate how to use the library's features.



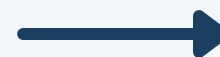
Esmaeel Moradi
@esmaeel-moradi700



A good example of a well-designed Kotlin library is Retrofit. It uses interfaces, annotations, and extension functions to provide a clean, type-safe API for network communication without requiring users to deal with the underlying implementation details.



Esmaeeil Moradi
@esmaeeil-moradi700



Architect Level Questions



Esmaeel Moradi
@esmaeeil-moradi700



Architect Kotlin-based domain-specific languages for business logic.

Architecting a full-fledged DSL for a complex business domain, such as a rule engine, involves more than just language features. It's about designing a system that is readable, maintainable, and testable.



Esmaeel Moradi
@esmaeeil-moradi700



1. Language Design:

- **Vocabulary:** The DSL should use the terminology of the business domain, not technical jargon.
- **Structure:** The DSL's structure should mirror the logical flow of the business rules. For example, a rule engine DSL might have a when block followed by a then block.

2. DSL Architecture:

- **Abstract Syntax Tree (AST):** The DSL should be parsed into an AST. This provides a structured representation of the business rules.
- **Interpreter:** An interpreter should then process the AST to execute the rules. This decouples the DSL's syntax from its execution logic.
- **Separation of Concerns:** The DSL should be purely for expressing business logic. The execute or apply logic should live outside the DSL itself, in the interpreter.



Esmaeel Moradi

@esmaeel-moradi700



3. Making the DSL Testable:

- **Unit Testing:** The DSL's builder functions should be unit-tested to ensure that they correctly construct the AST.
- **Integration Testing:** You should have integration tests that feed the DSL with real business rules and verify that the interpreter produces the correct output.
- **DSL DSL:** You can even use a "DSL for testing the DSL" to create highly readable tests.

4. Maintainability and Readability by Non-Developers:

- **Clarity:** The DSL's syntax should be so clear that a business analyst or a domain expert can read and understand the rules.
- **Tooling:** You should provide tools (e.g., a simple IDE) to help non-technical users write and manage the rules.
- **Documentation:** The DSL's reference documentation should be a central part of the system.



Esmaeel Moradi

@esmaeel-moradi700



This approach allows a business to manage its own rules, reducing the dependency on developers and increasing the speed of business logic changes.



Esmaeeil Moradi
@esmaeeil-moradi700



Design Kotlin performance optimization strategies for large applications.

For large applications, a holistic approach to performance optimization is necessary. It involves not just micro-optimizations but also architectural decisions and tooling.



Esmaeel Moradi
@esmaeeil-moradi700



1.Static Code Analysis with Tools:

- **Detekt:** Use static analysis tools like Detekt with a custom rule set to identify common performance pitfalls. For example, you can create a rule that flags the use of reflection in performance-critical code or flags excessive object allocation in loops.
- **Lint:** Create custom Android Lint rules to check for performance issues specific to your application.

2.Object Allocation Overhead:

- **Value Classes:** Use value classes (@JvmInline) to wrap primitive types. This eliminates object allocation for simple, type-safe data types.
- **Avoid Creating Unnecessary Objects:** Be mindful of creating objects in tight loops or in the onDraw method of a View. For example, a lambda that is not inlined creates an object on every call.
- **Object Pooling:** For frequently created objects, consider using object pools to reuse them and reduce garbage collection pressure.



Esmaeel Moradi

@esmaeel-moradi700



3. Concurrency and Parallelism:

- **Coroutines:** Use Kotlin coroutines to manage concurrency efficiently. Dispatchers.IO and Dispatchers.Default are optimized for I/O and CPU-intensive tasks, respectively.
- **Non-Blocking I/O:** Use non-blocking I/O with coroutines and Flow to prevent thread starvation.

4. Compilation and Build Performance:

- **Modularization:** The best way to improve build performance is through a well-designed modular architecture.
- **Gradle Build Cache:** Ensure the Gradle build cache is enabled and used effectively, both locally and remotely.



Esmaeeil Moradi

@esmaeeil-moradi700



The key to performance optimization is to measure before optimizing. Use profilers (e.g., Android Studio Profiler, perf on Linux) to find the actual bottlenecks and then apply the appropriate optimization strategy.



Esmaeel Moradi
@esmaeeil-moradi700

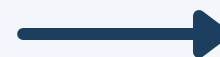


Create Kotlin code analysis and optimization tools.

Creating a code analysis and optimization tool for Kotlin requires using the Kotlin Compiler API or other libraries that can parse and inspect Kotlin code. This is an architect-level task that goes deep into the language's internals.



Esmaeel Moradi
@esmaeeil-moradi700



1. Using the Kotlin Compiler API:

- **K2 Compiler API:** The new K2 compiler provides a powerful API for inspecting and modifying the compiler's output. You can write a compiler plugin that can analyze the AST and report on patterns.
- **Parsing Code:** You can use the KotlinParser from the compiler API to parse Kotlin source files into an AST.



Esmaeel Moradi
@esmaeel-moradi700



2. Creating a Custom Lint Rule:

- **Lint API:** Android's Lint API allows you to create custom rules that check for specific patterns in Android projects.
- **Process:**
 - a. Create a custom Detector class that implements `com.android.tools.lint.client.api.Detector.JavaPsiScanner`.
 - b. Implement get methods to tell Lint what elements to visit (e.g., `UastMethod`, `UastClass`).
 - c. Implement a visit method to perform the analysis. For example, you can check if a specific, non-performance-critical function is called on the main thread.
 - d. Define an Issue and a Fix for the rule.



Esmaeel Moradi

@esmaeel-moradi700



3.What to Analyze:

- **Thread Safety:** Check for calls to network or disk I/O on the main thread.
- **Resource Management:** Check for close() calls on Closeable resources.
- **Performance:** Check for heavy computations in tight loops or view drawing code.
- **Code Style:** Enforce code style rules beyond what a formatter can do.



Esmaeel Moradi
@esmaeeil-moradi700



By creating these tools, you empower the entire organization to maintain a high standard of code quality and performance, enforcing best practices at scale.



Esmaeeil Moradi
@esmaeeil-moradi700

✨ These comprehensive answers aim to equip you with the knowledge required for Architect-level and Senior-level interviews, and to make informed architectural decisions in your real-world projects. Good luck. ✨



Esmaeeil Moradi

@esmaeeil-moradi700